PROGRAMMING

"Good programming," the eminent Dutch computer scientist Edsger Dijkstra snidely noted sometime in the 1970s, "is probably beyond the intellectual abilities of today's 'average programmer.'" Despite having computerized airline reservations, financial accounting, early warning systems, and countless other business and government functions, working programmers were deemed hopeless, for they had "been lured into a profession beyond their intellectual abilities." Such elitism was not universal. A few years earlier, in 1971, the inaugural issue of the People's Computer Company newsletter demanded a more demotic computing, with an image of a diverse protest of men, women, and children of many races on the march holding signs insisting "BASIC" is the people's language" and "Use computers *for* people, not against them."

Computer programming often exemplifies a vision of creation in which a thinking being implements a mentalistic plan. The history of electronic computing in its early years largely comprises the failures of just such a vision of labor and creative work. Computer programming involves iterative work with machines, work that fits easily neither as labor nor as ideation, even when that work involves deployment of sophisticated logical and mathematical forms. Both the image of programming as fundamentally pure thought stuff and the quotidian reality of its implication are historically, sociologically, and legally crucial. Programming steadfastly cannot be reduced either to math or to labor practices; as an object of study, it proves challenging for intellectual and labor historians in equal measure.

HUBRIS

Pioneers in the 1940s briefly believed that programming the new electronic *digital computers would be as simple as communicating logical and mathematical operations to coders to implement. "It had not occurred to me that there was going to be any difficulty about getting programs working," the English computer pioneer Maurice Wilkes quipped. "And it was with somewhat of a shock that I realized that for the rest of my life I was going to spend a good deal of my time finding mistakes that I had made in my programs." By the 1950s, the craft-like qualities of most industrial programming led to the understanding of the activity, in the words of the IBM researcher John Backus, as "a black art, a private arcane matter." Programming is a skilled intellectual craft, and it is, for any project of reasonable complexity, a social effort, demanding constant intercommunication—an effort resistant to hierarchy and standardization. In a major polemic entitled the *Mythical Man-Month*, Frederick Brooks argued, "Men and months are interchangeable commodities only when a task can be partitioned among many workers with no communication among them. . . . This is true of reaping wheat or picking cotton; it is not even approximately true of systems programming." Unlike many other

industries, programming does not scale by throwing more people at a programming task; expansion willy-nilly is likely to promote, not foreclose, failure.

In the 1960s, computer scientists drew on logic and algebra to refashion programming, culminating, as Mark Priestley describes, in "a move away from a step-by-step approach to computation to one based on higher-level operators which were described in terms of their overall effect." This shift altered fundamentally both the control structures of programs and the ways that large databases were envisioned and manipulated. While moving programming further from coding in machine language, this mathematical infusion did not result in programming being less a craft.

From the 1950s onward, a passel of programming philosophies has waxed and waned, and a panoply of management philosophies have been celebrated, tried, and found wanting. Typically, new solutions arose from a diagnosis of a crisis in programming, an intellectual, a social, or a business crisis—often a tangle of all three. Diverse social groups of programmers and managers have diagnosed the failures of programming in radically different ways. For academic computer scientists, such as Dijkstra, programming was inadequately mathematical and scientific; for commercial computing, programming was inadequately managed and organized. From structured programming to software engineering, no solution has proven adequate, whether for academic, scientific, or industrial needs; plenty involve a simple rebranding of earlier business philosophies; some reek of academic condescension and imperialism; some involve dramatic reworkings of thinking and business practices; most remain aspirational; each proposes, in varied measures, combinations of social and technological solutions to the collective creation of functioning software systems.

Drawing on "lean" manufacturing approaches, prominent efforts in the last twenty-five years have promoted iterative and incremental approaches rather than top-down bureaucratic and mathematical design philosophies and work processes. In 2001, to take a still influential example, seventeen software practitioners came together to *canonize a "Manifesto for Agile Software Development" centering on iteration in business and coding cycles alike, drawing on a variety of practices stretching back to the 1950s. "Through this work," they argued, "we have come to value":

> Individuals and interactions over processes and tools
> Working software over comprehensive documentation
> Customer collaboration over contract negotiation
> Responding to change over following a plan

These values in turn undergirded twelve principles intertwining the individual motivation and skills of programmers, their social organization, and their relationships to customers. "Working software is the primary measure of progress." Rather than organization from above, the "best architectures, requirements, and designs emerge from self-organizing teams." Each claim resonated against earlier bureaucratic and industrial orders of software design and spoke to developers in the flatter corporate structures of the early twenty-first century.

Initially far from corporate life, advocates of "free" software—free as in freedom, not free as in beer—sought technical architectures for collaborative programming that promoted adaptability and creativity rather than planning, and that substituted collective self-organization for top-down control. "Whereas controlled design and hierarchi-

cal planning represent the domain of governance—control through goal setting and orientation of a collective of a project—adaptability privileges politics," the anthropologist Chris Kelty notes, "properly speaking, the ability to critique existing design and to propose alternatives without restriction."

For all the attention to practices for creating new software, a vast edifice of legacy *code, often in old-fashioned languages such as COBOL, has long demanded and continues to demand maintenance. No code is every truly finished. "Because maintenance does not involve design," Nathan Ensmenger notes, "it was (and is) generally considered routine and low-status activity," even as it commands something estimated to be 60–80 percent of budgets for programming. Open source programming and agile programming, in never-ending development, often elides any distinction between discrete design and maintenance stages.

PROGRAMMING PEOPLE

Before computers were machines, computers were people performing calculations, and many of those people were women. During World War II, such women operated early mechanical calculating machines in computing ballistic trajectories, for example, and then became the first electronic computer operators in the *cryptographic effort at Bletchley Park in the UK and the scientific calculations of ENIAC in the USA. The division of labor predictably followed a gendered partition of origination and implementation. The percentage of women programmers has waxed and waned since, and in recent years the low numbers of women in the field in the United States has been an object of concern. Despite, or perhaps because of, their utopian pretensions, programming cultures outside corporations and academia have proven even worse for women. The open source and free software communities have had very few women indeed: in the early 2000s, around 1.5 percent of programmers as opposed to 28 percent in commercial software development. The causes remain up for debate; unquestionably, the collaborative culture of open source draws heavily on norms of masculine sociability, including the most toxic, from flame wars to the denunciation of newbies. And yet, the gendered quality of programming is hardly universal: in India, women made up 42 percent of computer science and engineering students in 2011, and, as Roli Varma and Deepak Kapur have argued, programmers are not broadly understood to share naturally the interests and forms of sociability central to American geek culture.

BETWEEN MATERIALITY AND IMMATERIALITY

Punching cards, plug boards, and repetitive stress injury from too much typing: programming calculating machines and computers has always been a physical human practice. Software might be thought stuff, pure ideation, math in a machine, but the process of making it is anything but, and it always runs on computers drawing energy, prone to fail, and possessing distinct material affordances. Facets of this materiality, Matthew Fuller in the *Software Lexicon* argues, include "the particular characteristics of a language or other form of interface, . . . how its compositional terms inflect and produce certain kinds of effects such as glitches, cross-platform compatibility, or ease of sharing and distribution; how, through both artifact and intent, events can occur at the level of

models of user subjectivity or forms of computational power, that exceed those of preexisting social formatting or demand new figures of knowledge." Historians, media scholars, and anthropologists stressing such material embedding of all programming argue in surprising parallel to corporate lawyers and lobbyists who have sought for decades to demonstrate that software is far more than mathematics, in order to gain patent and other intellectual property protections. Even if they may misconstrue or misunderstand programming, arguments concerning the tangibility and intangibility of software undergird alternative visions of the generation of knowledge and technology production, including those that challenge the intellectual property regimes in place since the nineteenth century.

THE "PROGRAMMER AS NAVIGATOR"

In 1973, the business information visionary Charles Bachman proclaimed that data storage was undergoing a "Copernican Revolution" that "is changing the programmer from a stationary viewer of objects passing before him in core into a mobile navigator who is able to probe and traverse a database at will." Rather than envisioning data as something examined serially, the advent of new random-access storage technologies like disk drives meant that digital computers could escape the traditions of serial information storage and retrieval inherited from early technologies of punch cards. Older technologies of retrieval, such as the index, featured elsewhere in this volume (see "indexing" and "storage and search"), could find analogs in digital technologies, if only programmers could shift their worldview. From this perspective, the task was to organize and store data for diverse programs and programmers to use.

A generation of development brought the database technologies, based primarily on the relational model, to the robustness and security that enabled global business throughout the end of the twentieth century. By the 1990s, many observers were announcing a crisis: programmers had figured out how securely to store data but they hadn't done nearly enough to provide tools for investigating that data.

COMPLEXITY, INSECURITY, AND TECHNICAL DEBT

By the late 1960s, consultants associated with the US National Security Agency (NSA) and the US Air Force began worrying about security issues associated with software systems, particularly as networks began to interconnect those systems. A 1970s report noted, "operating systems are very large, complex structures, and thus it is impossible to exhaustively test for every conceivable set of conditions that might arise." Complete design is impossible: "inadvertent loopholes exist" that "have not been foreseen by the designers." An attacker, this study predicted, "could mount a deliberate search for such loopholes with the expectation of exploiting them to acquire information." Computer systems quickly had proved far beyond any human capability to audit and to understand—even though the NSA tried, and tried, and tried. As with questions of maintenance, the complexity of existing software systems proved to be a gilded cage, a structure that human beings built that is outside of their control and understanding. Any software project of sufficient scope becomes for programmers, to borrow from Marx, "an alien force existing outside them, of the origin and goal of which they are ignorant, which

they thus cannot control, which on the contrary passes through a peculiar series of phases and stages independent of the will and the action of man, nay even being the prime governor of these." Our current condition of computer insecurity speaks to a complexity far outside our ken. "Program formulation is thus rather more like the creation of a bureaucracy," Joseph Weizenbaum wrote, "than like the construction of a machine" that can be comprehended as a whole.

Capital, if not *Kapital*, provided a metaphor now commonly used for understanding complexity of development and implementation of software systems. As the American developer of agile software Ward Cunningham wrote, "Shipping first time code is like going into debt. A little debt speeds development so long as it is paid back promptly with a rewrite." Dealing with debt is expensive—in future developer time. "The danger occurs when the debt is not repaid. Every minute spent on not-quite-right code counts as interest on that debt. Entire engineering organizations can be brought to a stand-still under the debt load of an unconsolidated implementation, object-oriented or otherwise."

FROM AUTOMATIC PROGRAMMING TO MACHINE LEARNING

"More and more, computers will program themselves," Herbert Simon predicted in 1961, "and direction will be given to computers through the mediation of compiling systems that will be completely neutral so far as the content of the decision rules is concerned." From the 1960s onward, ever more powerful tools enabled human beings to program at higher levels of abstraction from the physical hardware of machines, but the vision that computers might program themselves has proven elusive, both as an industrial practice and as project of scientific hopes. These dreams powerfully promoted, early on, the *conversion of mathematical language into higher-level languages without having to machine code oneself. Complaining of the "dull labor of writing and checking programs," Admiral Grace Hopper, pioneer of compilers, envisioned that the "programmer may return to being a mathematician." She was too sanguine. Generations of programming tools permit ever higher level work, but programmers have never returned to being mathematicians.

In his epochal 1950 paper on machine intelligence, Alan Turing noted that human beings will have programmed a computer to start acting intelligently just at the moment it ceases to act mechanically: "most of the programs which we can put into the machine will result in its doing something that we cannot make sense of at all, or which we regard as completely random behavior. Intelligent behavior presumably consists in a departure from the completely disciplined behavior involved in computation, but a rather slight one, which does not give rise to random behavior, or to pointless repetitive loops."

Producing computer systems capable of programing themselves in ways that appear to emulate or reproduce human intelligence has long been a desideratum of *artificial intelligence and has remained elusive—productively so. Those seeking to create computers capable of reasoning or emulating human reasoning have found themselves confronted with the difficulty of translating human knowledge and skills of all kinds into fully articulated descriptions capable of being programmed in the form of explicit rules. In the 1970s, the problem was diagnosed as the "knowledge-acquisition bottleneck,"

which underscored the same ineffability of skill as did the sociology of scientific knowledge of Harry Collins. In the early 1980s, the machine learning pioneer Donald Michie explained, "The inductive learning of concepts, rules, strategies, etc. from *examples* is what confers on the human problem-solver his power and versatility, and not (as had earlier been supposed) his powers of calculation." Today inductive algorithms trained on large data sets predict the judgments of human reasoners with no pretense of reproducing the thought processes and somatic judgments of human beings. Machine learning researchers have been most successful in creating programs capable of mimicking the outcomes of expert judgment precisely as they abandoned the effort to make computers proceed through hard-coded logical procedures and ceased to be priests of the high church of symbolic logic. Replicating decision making with the help of statistics, not the subordination of reasoning to logic and mathematics, fuels the ethical and political dangers of the explosion of machine learning all around us.

Using the example of the development of linear perspective in the art of the European *Renaissance, Dijkstra described how the "virtuoso coder" of the 1960s "feels like the medieval painter that could create a masterpiece whenever his experience enabled him to render proportions well, who suddenly found himself overtaken by all sorts of youngsters, pupils of Albrecht Dürer and the like, who had been taught the mathematical constructions that were guaranteed to surpass his most successful, but intuitive, renderings." The coming of mathematics to painting and sculpture provided a good analogy for the development of programming, but not for the reasons Dijkstra gave. Explaining the quality of Renaissance perspective in practice, the art historian James Elkins explains, "Laws, examples, and rules of thumb commingled, and there was a welter of more-or-less independent methods rather than a single mathematical set of axioms and a logically antecedent set of applications." So too with programming in the last half century. However much mathematically inspired techniques reshaped programming, they never could substitute for the welter of practices and knowledge undergirding our software-mediated world, any more than the tool kit of linear perspective dictated the content of paintings.

Matthew L. Jones

See also cards; computers; data; databases; indexing; intellectual property; networks; storage and search

FURTHER READING

Janet Abbate, *Recoding Gender: Women's Changing Participation in Computing*, 2012; Barry Boehm, "A View of 20th and 21st Century Software Engineering," in *Proceedings of International Conference on Software Engineering 2006*, 2006, 12–29; Wendy Hui Kyong Chun, *Programmed Visions: Software and Memory*, 2011; Geraldo Con Diáz, *Software Rights: How Patent Law Transformed Software Development in America*, 2019; Nathan Ensmenger, *The Computer Boys Take Over: Computers, Programmers, and the Politics of Technical Expertise*, 2010; Christopher M. Kelty, *Two Bits: The Cultural Significance of Free Software*, 2008; Mark Priestley, *A Science of Operations: Machines, Logic and the Invention of Programming*, 2010; Joy Lisi Rankin, *A People's History of Computing in the United States*, 2018.